

Project Documentation

Project Title

Secure Password Generator

Objective

To develop a Python application that generates secure passwords using cryptographically secure randomness and modern password security guidelines.

Problem Statement

Users often create weak passwords that are easy to guess. This project generates strong passwords containing letters, numbers, and special symbols to improve account security.

Technologies Used

- Python 3
- secrets module
- string module

Algorithm

1. Import required modules.
2. Create a character pool using:
 - `ascii_letters`
 - `digits`
 - `punctuation`
3. Ask the user to enter a password length.
4. Validate that the length is between 15 and 64.
5. Generate a password by repeatedly selecting random characters using `secrets.choice()`.
6. Combine the characters using `.join()`.
7. Display the generated password.

NIST SP 800-63-4 Considerations

Minimum Length

The program enforces a minimum password length of 15 characters.

Maximum Length

The program allows passwords up to 64 characters.

Secure Randomness

Uses `secrets.choice()` instead of `random.choice()` to provide cryptographically secure password generation.

Character Diversity

Passwords may contain:

- Uppercase letters
- Lowercase letters
- Numbers
- Special characters

Time Complexity

$O(n)$

Where n is the password length.

Space Complexity

$O(n)$

Where n is the password length.

Conclusion

The project successfully generates secure passwords using modern security practices and Python's cryptographically secure random number generation facilities.